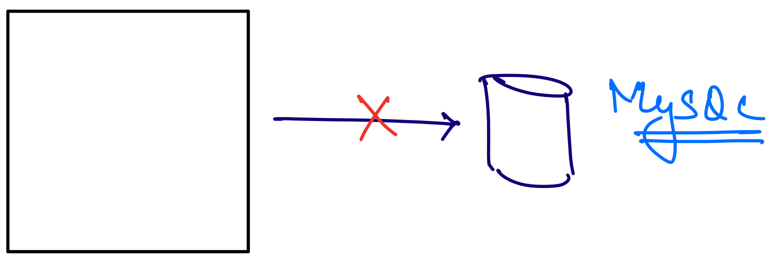
Agenda.

- → Introduction to JPA.
  - ↳ Java Persistence API.
  - → ORM ⇒ Hibernate
  - → JDBC
- → Connect our Product Service with DB.
  - ⇒ MySQL



⇒ Tight Coupling

- → Connect with DB.
- → CRUD operations on Entities.

Class Product {

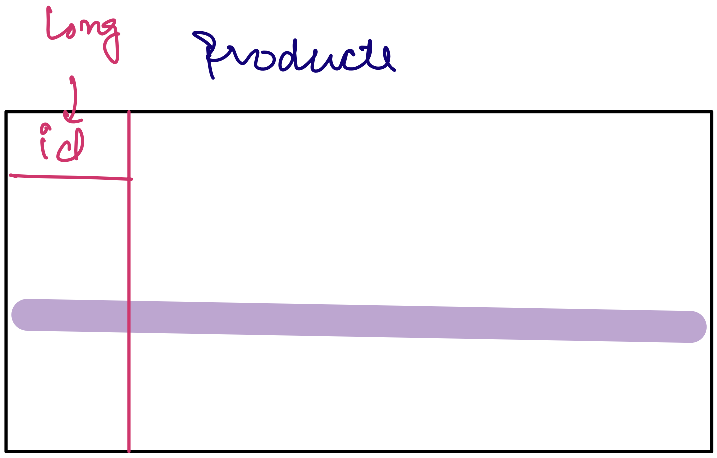
Long id

title

description

price

⇒



&

Class Diagram ≡ Schema Design

ORM

↳ Object Relational Mapping

Object ↓ Model

Relational ↓ Table

⇒ ORM provides us way to work with databases based on the models that we have in our Codebase.

→ Hibernate

→ MyBatis

→ Jooq

1 1

Product — Category. → M:1

M 1

→ Creation of tables.

→ Cardinalities b/w the Entities.

→ CRUD.

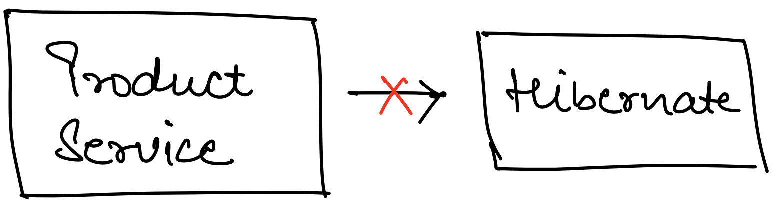
```
Select *
from Product
where id = ?
```

↓

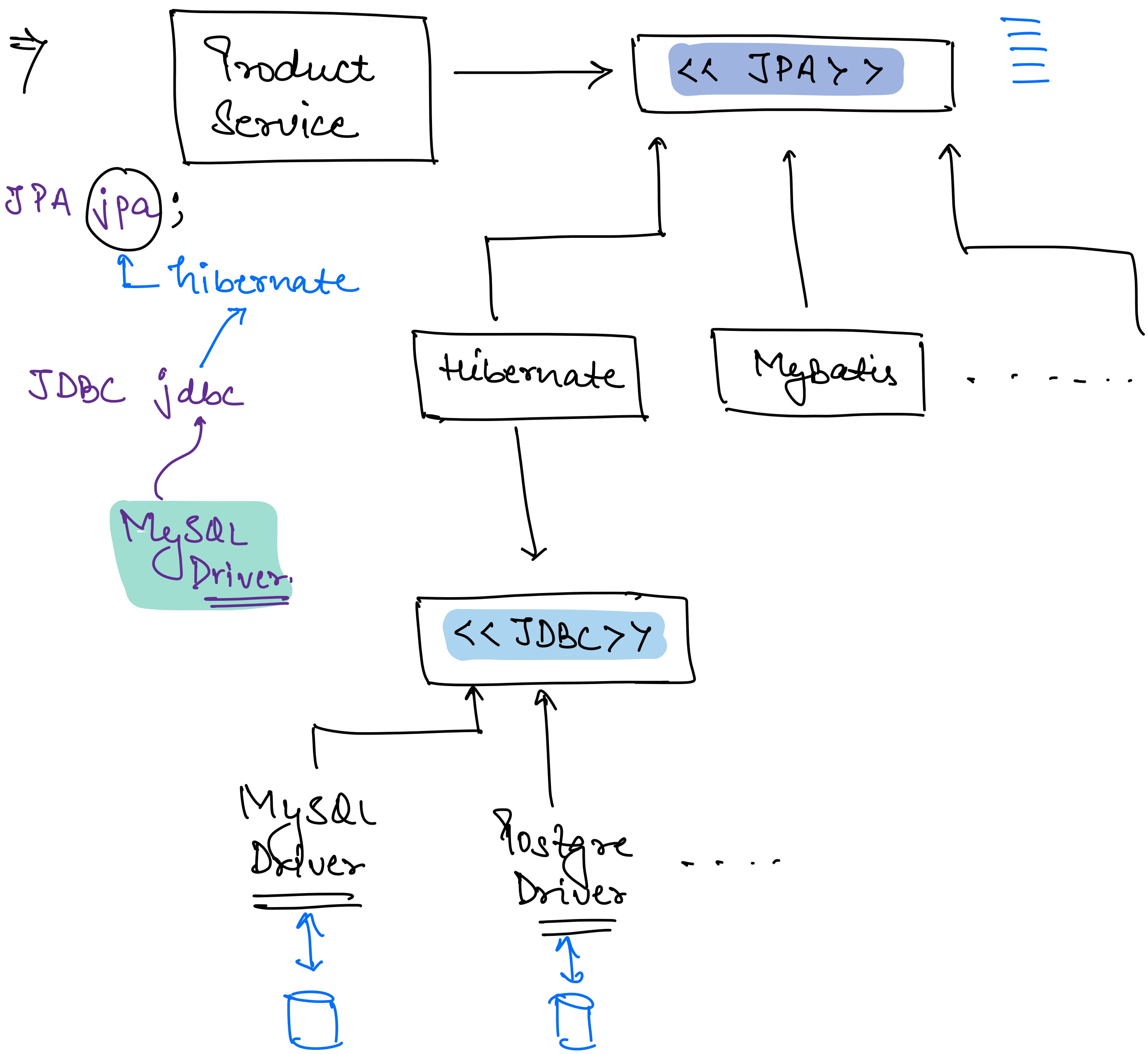
getProductBy(Id)

deleteProductBy(Id)

getAllProducts() ⇒



⇒ Tight Coupling



App → JPA → Hibernate → JDBC

↑

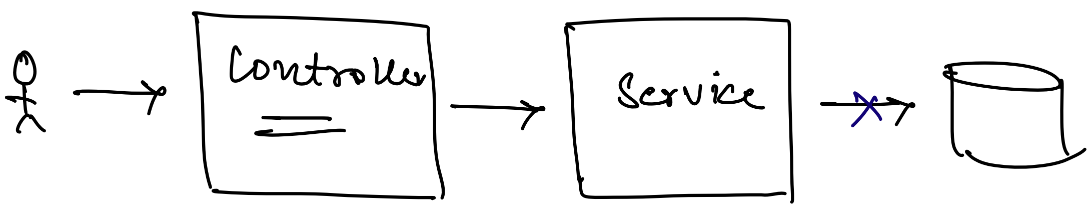
Driver

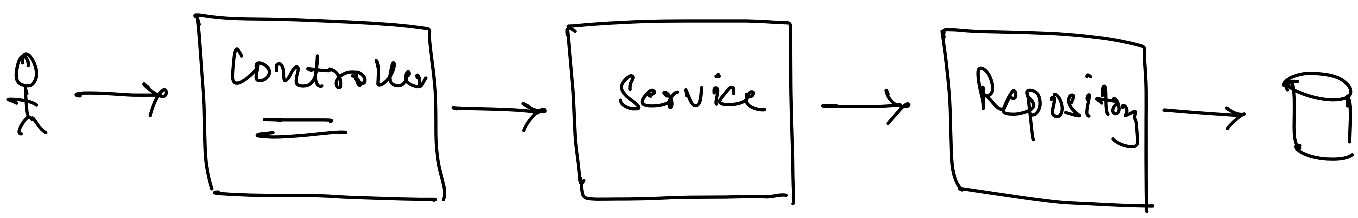
Spring Data JPA + Driver

[JPA + Hibernate]

⇒ Repository Pattern

Code to interact with persistence layer should be separate from Service layer.





Primary Key.

users

email	name	password .....

PK

1) User can change their email Id.
2) Indexing takes more space in case of string PK also indexing will become relatively slower.

⇒ Ideally, user attr & string attr shouldn't be used as a Primary key.

⇒ PK should be generated by the System.

1) Integer
   - → 4B
   - → Range.

   → **Auto Increment**

youtube.com/videos/1

2) long

→ 8B.

→ Auto Increment

Scaler.com/users/id

3) UUID. ⇒ 128 Bit Binary No.

d697935e-d72b-443b-ae5d-9aedb82909f1 ⇒ ~~STRING~~

$$\underbrace{\overset{5}{0101}\ \overset{3}{0011}\ \overset{2}{0010}\ \overset{C}{1100}\ 1010\ 0010 \ldots}_{128 \text{ no's}}$$

$$\frac{128}{4} = 32$$

→ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

10 → A

11 → B

12 → C

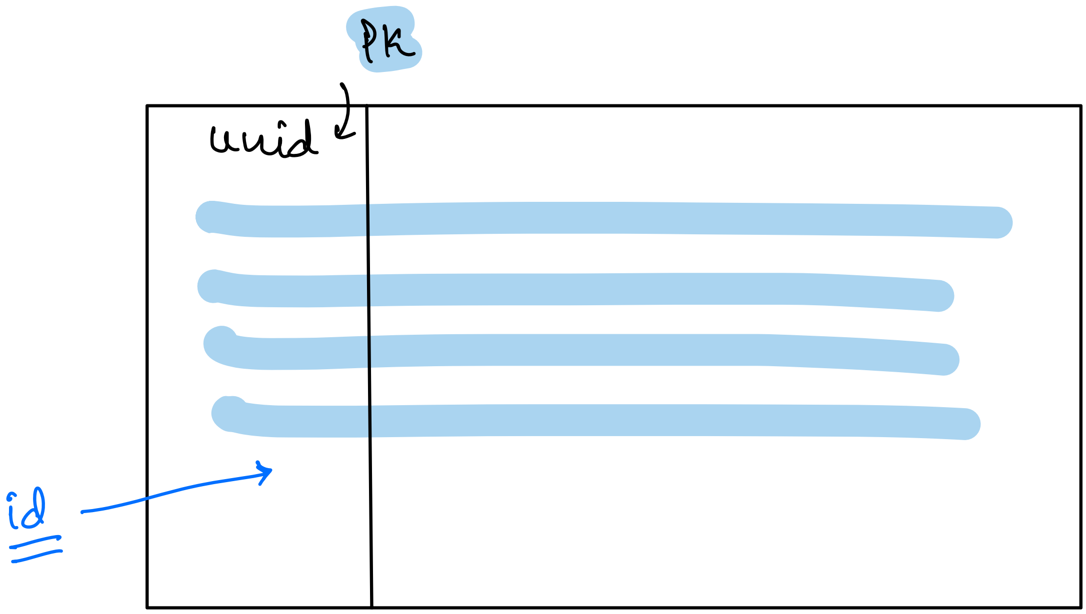
13 → D

14 → E

15 → F

$$\Rightarrow \quad \begin{array}{ccc} id_1 & \qquad & id_2 \\ \downarrow & & \downarrow \\ t_1 & & t_2 \end{array}$$

$$\text{if } (t_2 > t_1) \Rightarrow id_2 > id_1$$



UUID V7 ⇒ timestamp.

—— * ——